

Лабораторная работа 1

Часть 1: Введение в сверточные нейронные сети (CNN) для классификации изображений

Цель: Научиться основам построения и обучения модели CNN для классификации изображений, используя PyTorch. Изучить архитектуру CNN, включая сверточные слои, пулинг и активационные функции.

План работы:

1. Изучить базовую архитектуру CNN и её компоненты.
2. Подготовить датасет CIFAR-10 для классификации изображений.
3. Определить архитектуру сети с использованием сверточных и пулинг слоев.
4. Обучить модель и оценить её точность.

Задачи:

1. Загрузить датасет CIFAR-10, выполнить нормализацию и подготовить данные для обучения.
2. Определить архитектуру сети с двумя сверточными слоями, пулингом и полносвязными слоями.
3. Написать тренировочный цикл, запустить обучение и отслеживать точность на обучающей и тестовой выборках.
4. Визуализировать функцию потерь и оценить точность, объяснив, какие улучшения можно внести для повышения производительности.

Часть 2: Классификация изображений с предобработкой и аугментацией данных

Цель: Понять влияние предобработки и аугментации данных на работу модели CNN. Изучить, как различные типы данных и аугментация помогают модели обобщать на разных выборках, таких как CIFAR-10, Fashion-MNIST и SVHN.

План работы:

1. Подготовить данные из нескольких датасетов, применить разные методы аугментации (горизонтальное отражение, повороты, яркость).
2. Обучить модель на каждом наборе данных и сравнить результаты.
3. Проанализировать, как аугментация влияет на обобщающую способность модели.

Задачи:

1. Загрузить и предобработать CIFAR-10, Fashion-MNIST и SVHN, применяя различные техники аугментации.
2. Обучить модель CNN на каждом из датасетов, сравнить точность и функцию потерь.
3. Оценить влияние аугментации на результат и описать, какие методы лучше подходят для каждого датасета.
4. Сделать выводы о том, как предобработка помогает улучшить обобщающие способности модели.

Часть 3: Оптимизация архитектуры CNN

Цель: Изучить влияние изменения количества и размера фильтров, пулинг слоев, Batch Normalization и Dropout на производительность модели CNN. Оценить, как эти изменения влияют на переобучение и обобщающую способность модели.

План работы:

1. Создать несколько вариантов CNN с разными архитектурными параметрами (количество фильтров, размеры ядер, использование Batch Normalization и Dropout).
2. Обучить и сравнить производительность моделей.
3. Понять, как архитектурные параметры влияют на обобщение и точность модели.

Задачи:

1. Создать три варианта архитектуры CNN с разным количеством фильтров и размерами ядер.
2. Добавить и настроить Batch Normalization и Dropout для предотвращения переобучения.
3. Провести сравнение моделей по точности и функции потерь на тестовой выборке.
4. Описать, какой подход лучше работает для данной задачи и почему, предлагая возможные улучшения.

Часть 4: Оптимизация гиперпараметров с использованием нестандартных методов

Цель: Изучить влияние гиперпараметров, таких как learning rate, batch size и количество слоев. Применить методы циклического темпа обучения, адаптивного learning rate, динамического изменения batch size и анализировать, как эти гиперпараметры влияют на обучение.

План работы:

1. Настроить базовый learning rate и batch size, применить Cyclic Learning Rate и Adaptive Learning Rate.
2. Изучить влияние batch size и количества слоев на обучение.
3. Сравнить результаты по разным стратегиям и проанализировать, какая оптимизация работает лучше всего.

Задачи:

1. Обучить модель с циклическим и адаптивным темпами обучения, сравнить результаты.
2. Протестировать разные значения batch size и настроить накопление градиентов для экономии памяти.
3. Сравнить модели с разным количеством фильтров и слоев, чтобы оценить влияние каждого гиперпараметра.
4. На основе полученных результатов предложить оптимальные настройки гиперпараметров для повышения точности и стабильности обучения.

Лабораторная работа 1: Классификация изображений с использованием CNN, предобработка и аугментация на различных датасетах, анализ метрик**Раздел 1: Подготовка данных и работа с несколькими датасетами**

Цель

Изучить влияние различных типов данных и предобработки на качество работы CNN.

Теория

Для глубокого понимания устойчивости модели и способности к обобщению полезно использовать несколько датасетов с разными характеристиками, такими как:

- CIFAR-10: Небольшие цветные изображения с разнообразными классами (животные, объекты).
- Fashion-MNIST: Черно-белые изображения одежды, подходящие для классификации текстур и форм.
- SVHN: Цветные изображения цифр из уличных знаков, что помогает оценить устойчивость модели к вариативности и сложным фонам.

1. Загрузка и предобработка данных для каждого датасета:

```
from torchvision import datasets, transforms
```

```
transform_cifar = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
transform_fmni = transforms.Compose([
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

```
transform_svhn = transforms.Compose([
    transforms.RandomVerticalFlip(),
    transforms.ColorJitter(brightness=0.2),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
cifar10_train = datasets.CIFAR10(root='data', train=True, transform=transform_cifar,
download=True)
fmnist_train = datasets.FashionMNIST(root='data', train=True,
transform=transform_fmni, download=True)
svhn_train = datasets.SVHN(root='data', split='train', transform=transform_svhn,
download=True)
```

2. Сравнение предобработки для разных типов данных:

- Использовать каждый метод предобработки с каждым датасетом и обучить модель, чтобы увидеть влияние на устойчивость и обобщающую способность модели.
- Анализировать, как аугментация, такая как горизонтальное и вертикальное отражение, яркость и повороты, улучшает или ухудшает результаты.

На что обратить внимание:

- Слишком агрессивная аугментация может ухудшить качество, если она приводит к искажению важных признаков.
- Убедиться, что предобработка улучшает производительность на каждом конкретном датасете – если нет, возможно, аугментацию следует изменить.

3: Создание и оптимизация архитектуры CNN: Теория и Практика

В этом разделе мы рассмотрим, как изменять архитектуру CNN и оптимизировать её.

Компоненты архитектуры CNN и их влияние

Базовый код для построения и тренировки CNN-модели

1. Импорт необходимых библиотек

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

2. Подготовка данных

Используем CIFAR-10 – популярный датасет для классификации изображений, содержащий 10 классов.

Трансформации для нормализации изображений

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

Загрузка тренировочного и тестового наборов данных

```
train_data = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
test_data = torchvision.datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)
```

Создание загрузчиков данных

```
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
```

```
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)
```

3. Определение базовой архитектуры CNN

Эта простая сеть состоит из двух сверточных слоев с пулингом, слоем нормализации и полносвязными слоями для классификации.

```
class SimpleCNN(nn.Module):
```

```
    def __init__(self):
```

```
        super(SimpleCNN, self).__init__()
```

```
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
```

```
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.fc1 = nn.Linear(64 * 8 * 8, 128) # 64 каналов на выходе после второго пулинга
```

```
        self.fc2 = nn.Linear(128, 10) # 10 классов для CIFAR-10
```

```
        self.dropout = nn.Dropout(p=0.5)
```

```
        self.batch_norm = nn.BatchNorm2d(64)
```

```
    def forward(self, x):
```

```
        x = self.pool(torch.relu(self.conv1(x)))
```

```
        x = self.pool(self.batch_norm(torch.relu(self.conv2(x))))
```

```
        x = x.view(-1, 64 * 8 * 8)
```

```
        x = self.dropout(torch.relu(self.fc1(x)))
```

```
        x = self.fc2(x)
```

```
        return x
```

Инициализация модели

```
model = SimpleCNN()
```

4. Настройка функции потерь и оптимизатора

Используем CrossEntropyLoss для классификации и Adam для оптимизации параметров.

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

5. Тренировочный цикл

Теперь создадим цикл тренировки с функцией потерь и отслеживанием точности на каждом шаге.

```
num_epochs = 10
```

```
train_losses = []
```

```
test_losses = []
```

```
for epoch in range(num_epochs):
```

```

model.train()
running_loss = 0.0
for images, labels in train_loader:
    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.item()
train_losses.append(running_loss / len(train_loader))
print(f"Эпоха {epoch+1}/{num_epochs}, Потеря на обучении: {train_losses[-1]:.4f}")

# Оценка на тестовых данных
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f"Точность на тестовом наборе: {accuracy:.2f}%")

```

6. Визуализация функции потерь

После тренировки можно построить график функции потерь, чтобы увидеть, как модель сходится.

```

plt.plot(train_losses, label='Потери на обучении')
plt.xlabel('Эпоха')
plt.ylabel('Потери')
plt.legend()
plt.show()

```

Оптимизация

1. Количество и размер сверточных фильтров

Сравним несколько вариантов слоев с разным количеством фильтров и размерами ядер, чтобы увидеть их влияние на производительность.

Пример 1: Использование меньшего количества фильтров (16 и 32) с ядрами 3x3 и 5x5

```
import torch.nn as nn
```

Слой с фильтром 3x3 и 16 каналами

```
conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
```

Слой с фильтром 5x5 и 32 каналами

```
conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, padding=2)
```

Пример 2: Увеличение числа фильтров (64 и 128) для более сложных признаков

Слой с фильтром 3x3 и 64 каналами

```
conv1 = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
```

Слой с фильтром 5x5 и 128 каналами

```
conv2 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=5, padding=2)
```

Пример 3: Использование фильтров с ядрами 7x7 и большим числом каналов для очень сложных признаков

Слой с фильтром 7x7 и 256 каналами

```
conv = nn.Conv2d(in_channels=3, out_channels=256, kernel_size=7, padding=3)
```

На что обратить внимание:

- Увеличение числа фильтров улучшает способность модели к распознаванию сложных признаков, но также увеличивает вычислительные затраты и риск переобучения.
- Увеличение размера ядра позволяет распознавать более крупные и обобщенные признаки, но может привести к потере мелких деталей.

2. Пулинг (Pooling) слои

Изучим влияние Max Pooling и Average Pooling на модель.

Пример 1: Max Pooling с размером 2x2, stride 2

```
maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
```

Пример 2: Max Pooling с большим размером ядра (3x3) для более сильного снижения размерности

```
maxpool = nn.MaxPool2d(kernel_size=3, stride=3)
```

Пример 3: Average Pooling для сглаживания признаков

```
avgpool = nn.AvgPool2d(kernel_size=2, stride=2)
```

На что обратить внимание:

- **Max Pooling** сохраняет наибольшие значения в окне и делает модель устойчивой к шуму, что особенно полезно для извлечения острых краев и контуров.
- **Average Pooling** сглаживает данные и лучше подходит для задач, где важны общие паттерны, а не отдельные пиксели.

3. Batch Normalization

Batch Normalization помогает стабилизировать и ускорить обучение, особенно в глубоких сетях.

Пример 1: Batch Normalization после каждого сверточного слоя

Конвейер с двумя слоями и Batch Normalization

```
conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
bn1 = nn.BatchNorm2d(32)
conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
bn2 = nn.BatchNorm2d(64)
```

Пример 2: Batch Normalization только в начале и конце сети

Batch Normalization только в начале и после последнего сверточного слоя

```
conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
bn1 = nn.BatchNorm2d(32)
conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1)
bn3 = nn.BatchNorm2d(128)
```

На что обратить внимание:

- **Batch Normalization** делает модель устойчивее к настройке learning rate и помогает ускорить сходимость.
- Влияние Batch Normalization уменьшается в небольших сетях, но становится более значимым в глубоких сетях.

4. Dropout

Dropout снижает переобучение, отключая случайные нейроны во время обучения.

Пример 1: Умеренный Dropout (0.2)

```
dropout = nn.Dropout(p=0.2)
```

Пример 2: Сильный Dropout (0.5)

```
dropout = nn.Dropout(p=0.5)
```

Пример 3: Комбинация разных значений Dropout в одном слое

```
dropout1 = nn.Dropout(p=0.3)
```

```
dropout2 = nn.Dropout(p=0.5)
```

На что обратить внимание:

- При сильном значении Dropout (например, 0.5) могут теряться важные признаки, что ухудшит точность.
- Dropout в пределах 0.2-0.3 часто лучше подходит для небольших сетей.

5. Полносвязные слои

Полносвязные слои классифицируют признаки, извлеченные сверточными слоями. Разные настройки нейронов в этих слоях могут по-разному повлиять на производительность.

Пример 1: Полносвязный слой с малым количеством нейронов

```
fc = nn.Linear(128, 10) # Для 10 классов
```

Пример 2: Увеличение количества нейронов для более сложной классификации

```
fc = nn.Linear(128, 100) # Для более сложной задачи классификации
```

Пример 3: Последовательное применение нескольких полносвязных слоев

```
fc1 = nn.Linear(128, 64)
```

```
fc2 = nn.Linear(64, 10) # Финальный слой для 10 классов
```

На что обратить внимание:

- Если слишком много нейронов, это может привести к переобучению.
- Последовательное использование нескольких полносвязных слоев помогает сжать признаки перед финальной классификацией.

Определение правильного направления оптимизации

1. Анализ функции потерь

```
import matplotlib.pyplot as plt
```

Предположим, что у нас есть массивы значений функции потерь на обучении и тестировании

```
loss_train = [2.0, 1.5, 1.2, 1.0, 0.8] # Пример значений функции потерь на обучении
```

```
loss_val = [2.2, 1.8, 1.5, 1.3, 1.1] # Пример значений функции потерь на тестировании
```

```
plt.plot(loss_train, label="Training Loss")
plt.plot(loss_val, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

2. Анализ точности на обучении и тестировании

```
accuracy_train = [0.6, 0.7, 0.8, 0.85, 0.88]
```

```
accuracy_val = [0.55, 0.68, 0.75, 0.8, 0.83]
```

```
plt.plot(accuracy_train, label="Training Accuracy")
plt.plot(accuracy_val, label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

На что обратить внимание:

- Функция потерь: Постепенное снижение указывает на успешное обучение. Если есть резкие колебания, уменьшите learning rate.
- Точность на обучении и тесте: Если точность на тесте не достигает значений на обучении, это может быть признаком переобучения.

Пример вывода метрик с визуализацией:

```
from sklearn.metrics import roc_auc_score, roc_curve
```

```
y_pred_proba = model.predict_proba(X_test)
roc_auc = roc_auc_score(y_test, y_pred_proba, multi_class='ovr')
print("ROC-AUC Score:", roc_auc)
```

```
# Визуализация ROC-кривой
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba[:,1])
plt.plot(fpr, tpr, label=f'ROC curve (area = {roc_auc:.2f})')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
```

Раздел 4: Анализ гиперпараметров и оптимизация с использованием нестандартных методов

В этом разделе будем изучать влияние гиперпараметров и использовать продвинутые методы оптимизации, такие как циклический темп обучения (Cyclic Learning Rate), адаптивный темп обучения (Adaptive Learning Rate), автоматическая настройка размера пакета (Dynamic Batch Size) и продвинутое тестирование количества фильтров и слоев для лучшего понимания того, как эти параметры влияют на обучение и производительность модели.

Основные гиперпараметры и их влияние на обучение

1. Learning Rate (Темп обучения)

- Теория: Learning Rate управляет тем, насколько сильно изменяются веса модели при каждом шаге оптимизации. Низкий learning rate замедляет обучение, но делает его стабильным, высокий learning rate ускоряет обучение, но может вызвать колебания и нестабильность.
- Методы оптимизации:

- Cyclic Learning Rate: Позволяет изменять темп обучения в ходе обучения, перемещаясь между заданным минимумом и максимумом. Это помогает выйти из локальных минимумов и может повысить производительность модели.
- Adaptive Learning Rate: Использует оптимизаторы, такие как Adam или RMSprop, которые автоматически корректируют learning rate на основе градиентов.

- Практические упражнения:

1. Использование Cyclic Learning Rate:

```
from torch.optim.lr_scheduler import CyclicLR
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

```
scheduler = CyclicLR(optimizer, base_lr=0.001, max_lr=0.01, step_size_up=5)
```

Задача: Обучить модель, наблюдая, как модель сходится при циклическом темпе обучения. Сравните это с постоянным темпом обучения.

2. Исследование адаптивного learning rate с Adam:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Задача: Запустите модель с оптимизатором Adam и сравните ее с SGD.

Наблюдайте, как Adam адаптирует learning rate для разных слоев и стабилизирует обучение.

- На что обратить внимание:
 - Если модель не сходится, возможно, нужно расширить диапазон Cyclic Learning Rate или уменьшить начальное значение в Adam.
 - Требуется сравнить функции потерь и точность для различных стратегий и выяснить, какая стратегия подходит лучше.

2. Batch Size (Размер пакета)

- Batch Size определяет количество примеров, используемых для одного шага обновления весов. Большой batch size увеличивает вычислительную эффективность, но может снизить способность модели обобщать. Малый batch size создает большее разнообразие, но увеличивает шум в градиентах.
- Методы оптимизации:
 - Dynamic Batch Size: Плавное увеличение batch size в процессе обучения помогает улучшить стабильность и позволяет модели сходиться быстрее на начальных этапах.
 - Gradient Accumulation: Используется при ограниченной памяти. Вместо увеличения batch size, градиенты накапливаются за несколько шагов, а затем обновляются веса.
- Практические упражнения

1. Динамическое изменение Batch Size:

```

initial_batch_size = 32
for epoch in range(num_epochs):
    if epoch % 5 == 0:
        current_batch_size = initial_batch_size * (epoch // 5 + 1)
        # Перезапуск DataLoader с новым batch_size
        train_loader = DataLoader(train_data, batch_size=current_batch_size, shuffle=True)

```

Задача: Изменяйте batch size каждые 5 эпох, наблюдая, как это влияет на сходимость. Попробуйте найти оптимальный шаг увеличения.

2. Gradient Accumulation:

```

accumulation_steps = 4
optimizer.zero_grad()
for i, (inputs, labels) in enumerate(train_loader):
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

```

Задача: Сравните результаты с разными значениями accumulation_steps и обычным большим batch size. Обратите внимание, как это влияет на расход памяти и сходимость.

- На что обратить внимание:
 - Если модель переобучается, уменьшение batch size может помочь повысить обобщающую способность.
 - Сравните влияние разных batch size на графики функции потерь и точности.

3. Количество фильтров и слоев

- Теория: Количество фильтров определяет, сколько признаков модель может извлечь на каждом уровне. Чрезмерное увеличение числа фильтров и слоев приводит к переобучению, но недостаточное – к недообучению и низкой способности модели к обобщению.
- Методы оптимизации:
 - Progressive Layer Growth: Добавление слоев по мере обучения, что позволяет начать с небольшой модели и увеличивать её сложность, если это необходимо.
 - Random Filter Pruning: Уменьшение количества фильтров, которые наименее активно используются, чтобы упростить модель и уменьшить переобучение.
- Практические упражнения:

1. Progressive Layer Growth:

- Начать с простой CNN и добавлять слои после нескольких эпох обучения.

```

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(16 * 16 * 16, 10) # Начальная модель

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)
        x = x.view(-1, 16 * 16 * 16)
        x = self.fc1(x)
        return x

```

Добавляем больше слоев после 10 эпох

```

class ExtendedCNN(SimpleCNN):
    def __init__(self):
        super(ExtendedCNN, self).__init__()
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc2 = nn.Linear(32 * 8 * 8, 10)

```

Задача: Наблюдайте, как изменение количества слоев влияет на точность. Студенты могут определить, какой размер модели лучше подходит для конкретного датасета.

2. Random Filter Pruning:

```

import torch.nn.functional as F

```

```

def prune_filters(model, threshold=0.1):
    for name, param in model.named_parameters():
        if "weight" in name:
            param.data = F.dropout(param.data, p=threshold)

```

```

prune_filters(model, threshold=0.2)

```

Задача: Установите threshold и удалите наименее активные фильтры, обучите модель и наблюдайте, как это влияет на производительность.

- На что обратить внимание:
 - Progressive Layer Growth может быть полезным, если модель не сходится с начальной архитектурой. Однако важно остановиться на оптимальном количестве слоев, чтобы избежать переобучения.
 - Pruning фильтров может улучшить производительность модели на тестовых данных, если модель демонстрирует признаки переобучения.

Подведение итогов и анализ

1. Сравнительный анализ стратегий оптимизации гиперпараметров:

- Необходимо проанализировать, какая стратегия лучше всего подходит для их задачи и почему.
 - Выяснить, какой метод оптимизации learning rate, batch size и числа фильтров наиболее эффективно улучшает точность и стабильность обучения.
2. Методические пояснения:
- Learning Rate: Cyclic и adaptive learning rate помогают выйти из локальных минимумов и ускорить обучение на сложных датасетах.
 - Batch Size: Динамическое изменение batch size или использование gradient accumulation полезны для улучшения сходимости при ограниченных ресурсах.
 - Количество фильтров и слоев: Progressive Layer Growth и Random Pruning позволяют адаптировать сложность модели к объему данных и предотвращать переобучение.

Раздел 5: Анализ различных метрик производительности

Цель

Научиться анализировать и интерпретировать различные метрики для оценки качества модели.

Помимо точности, в анализе производительности модели важно учитывать другие метрики, такие как:

- Precision: Указывает, насколько точно модель предсказывает положительные классы, минимизируя ложноположительные результаты.
- Recall: Определяет, насколько полно модель выявляет все истинные положительные примеры.
- F1-score: Гармоническое среднее Precision и Recall, подходит для несбалансированных классов.
- ROC-AUC: Показывает, насколько хорошо модель различает классы.

Практика

1. Вычисление и интерпретация метрик:

```
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score
```

```
precision = precision_score(y_true, y_pred, average='weighted')
```

```
recall = recall_score(y_true, y_pred, average='weighted')
```

```
f1 = f1_score(y_true, y_pred, average='weighted')
```

```
roc_auc = roc_auc_score(y_true, y_pred_proba, multi_class='ovr')
```

```
print("Precision:", precision)
```

```
print("Recall:", recall)
```

```
print("F1-score:", f1)
```

```
print("ROC-AUC Score:", roc_auc)
```

2. Сравнение метрик:

- Построить графики для каждой метрики на обучающем и тестовом наборах данных, чтобы определить, где модель работает лучше.

На что обратить внимание:

- Высокий Precision и низкий Recall указывают, что модель хорошо предсказывает позитивные примеры, но может пропускать некоторые из них.
- ROC-AUC подходит для многоклассовой классификации. Высокий показатель означает, что модель эффективно различает классы.

6. Сравнение и анализ результатов на разных датасетах и методах предобработки

- Оценка и визуализация метрик: Построить графики точности, функции потерь и других метрик для каждой модели и датасета. Сравнить и обсудить, как предобработка, аугментация и архитектура влияют на производительность модели.

```
import matplotlib.pyplot as plt
```

```
def plot_metrics(train_acc, val_acc, train_loss, val_loss):
```

```
    epochs = range(len(train_acc))
    plt.figure(figsize=(10,5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_acc, label='Train Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.legend()
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.title('Accuracy per Epoch')
```

```
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_loss, label='Train Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.legend()
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title('Loss per Epoch')
```

```
plt.show()
```

Методические пояснения:

- Предобработка и аугментация: Разные методы предобработки и аугментации работают по-разному с разными типами данных. Например, горизонтальное отражение полезно для CIFAR-10, но может не подойти для Fashion-MNIST.
- Анализ метрик: Каждая метрика имеет своё применение. Точность полезна для общего понимания модели, а F1-score важен при дисбалансе классов. Анализ различных метрик помогает понять, какие аспекты модели требуют улучшения.
- Эксперименты с гиперпараметрами: Ручные эксперименты с гиперпараметрами дают более глубокое понимание того, как каждый параметр влияет на обучение и качество модели.

